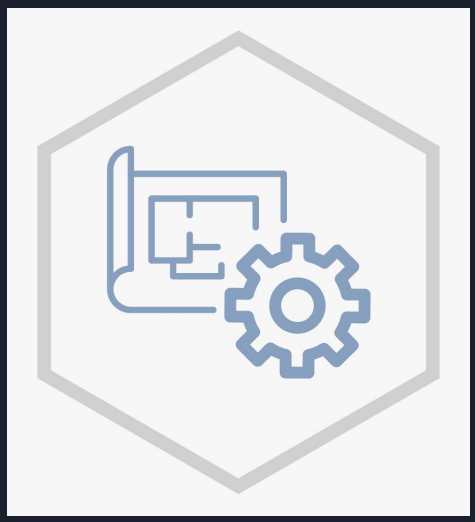
System Design



Name: Rohan Vashisht
Cohort: Jensen Huang
Roll No.: 150096724132

GitHub:
https://github.com/RohanVashisht1234/college-system-design

Colab:
https://colab.research.google.com/drive/1BXZRrdZ3ZTJDe93QXP8ldVA0SFM0LkgC?usp=sharing

System Design Diagram:

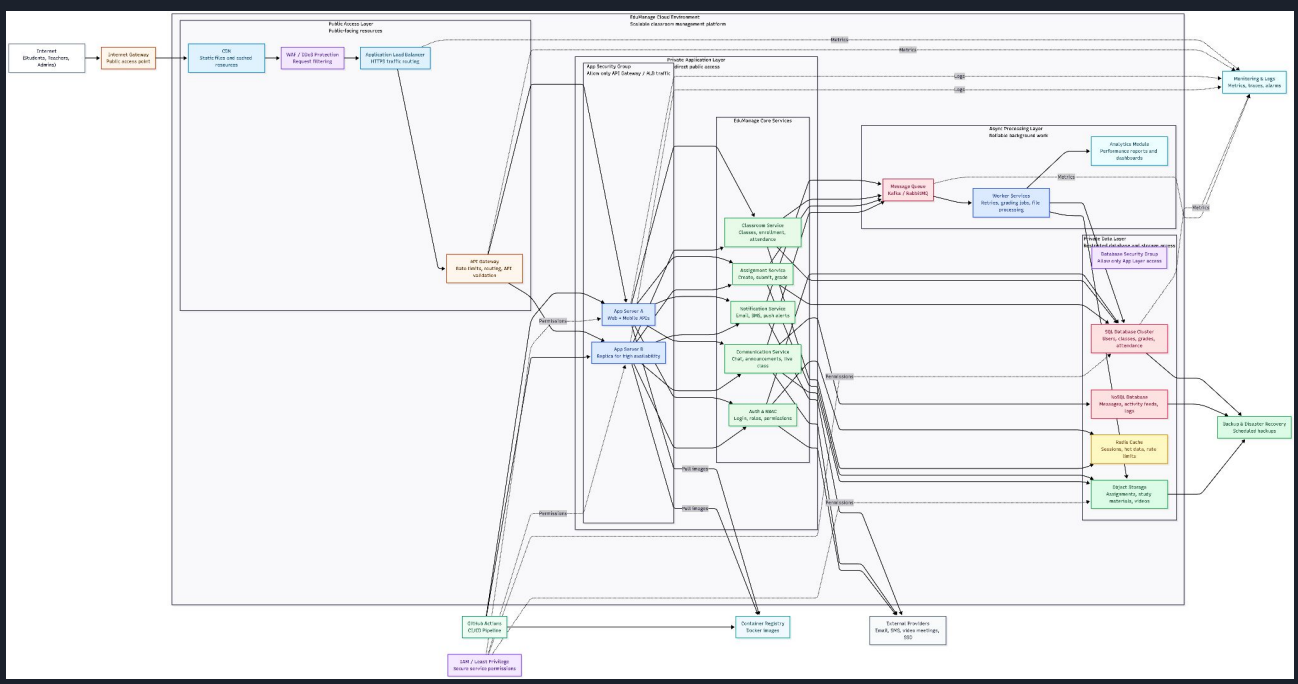


Table of Contents

1. Problem Statement

Modern educational institutions face growing challenges in managing classrooms, assignments, grading, communication, and attendance across thousands of students and teachers simultaneously. Legacy or manual systems suffer from poor scalability, data silos, and lack of real-time feedback loops.

Core Challenges

Scalability Bottlenecks

Traditional systems struggle when thousands of students submit assignments or attend virtual classes simultaneously, causing slowdowns and failures.

Data Fragmentation

Student records, grades, attendance, and communication logs are often scattered across incompatible systems with no unified access layer.

Security & Access Control

Role-based restrictions between students, teachers, and administrators are difficult to enforce consistently without a centralized auth system.

Asynchronous Workflows

Resource-intensive tasks like bulk grading, file processing, and report generation cannot block primary user interactions, yet many systems handle them synchronously.

Communication Delays

Students and teachers lack real-time channels for announcements, notifications, and feedback, degrading the learning experience.

Analytics Gaps

Administrators have limited insight into academic performance trends, attendance patterns, and engagement metrics at the institution level.

2. Proposed Solution

EduManage is a cloud-native, microservices-based classroom management platform designed to serve millions of users across thousands of institutions. It replaces fragmented legacy tools with a unified, secure, and scalable digital academic environment.

Key Objectives

Unified Platform Single entry point for students, teachers, and administrators with role-appropriate views.	**Horizontal Scalability** Load-balanced app servers and queue-based async workers handle traffic spikes gracefully.
Secure by Design WAF, RBAC, IAM least-privilege, and database security groups limit exposure at every layer.	**Real-time Communication** Notification and communication services push updates instantly via email, SMS, and push.
Reliable Background Processing Message queues decouple grading and file processing from user-facing requests.	**Data-Driven Analytics** Consolidated analytics module provides actionable academic insights.

Core Approach

The solution adopts a layered cloud architecture with clear separation between public-facing resources, application logic, data storage, and background processing. Each layer communicates only through well-defined interfaces, minimising the blast radius of any component failure.

- CDN and WAF absorb static traffic and filter malicious requests before they reach application servers.
- A load balancer distributes requests across replica app servers, providing high availability.
- An API gateway enforces rate limits and validates payloads before forwarding to internal services.
- Authentication and RBAC services gate every request, ensuring role-appropriate data access.
- Core microservices (Classroom, Assignment, Grading, Communication, Notification) are independently scalable.
- SQL, NoSQL, Redis, and object storage serve distinct data-access patterns optimally.
- Kafka/RabbitMQ queues decouple heavy tasks, and worker services process them asynchronously.
- CI/CD pipelines, container registries, IAM policies, and monitoring complete the operational picture.

3. System Architecture

Architecture Overview

The EduManage architecture is organised into four primary cloud layers plus external integrations. Every internet request passes through the Public Access Layer before touching application or data services, ensuring a defence-in-depth posture.

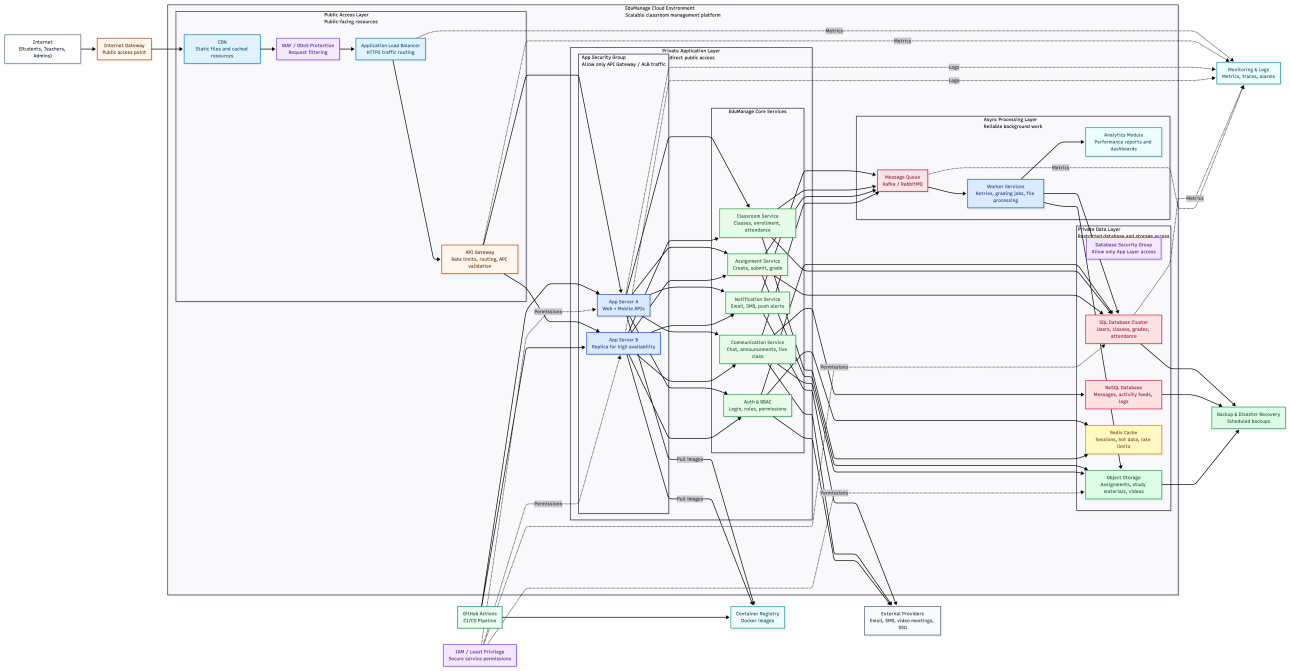


Figure 1 – EduManage System Architecture Diagram

Layer Breakdown

Public Access Layer

Internet Gateway → CDN → WAF/DDoS Protection → Application Load Balancer → API Gateway. This layer handles TLS termination, static-asset caching, malicious-request filtering, HTTPS routing, and API-level rate limiting. No direct database access is possible from here.

Private Application Layer

App Server A and App Server B (replica) sit inside a security group that accepts connections only from the API Gateway. They expose REST/WebSocket endpoints for web and mobile clients and fan out to core microservices. Horizontal scaling is achieved by adding more replicas behind the load balancer.

EduManage Core Services

Auth & RBAC, Classroom Service, Assignment Service, Communication Service, and Notification Service. Each service has a single responsibility and communicates with the data layer through parameterised queries, never raw SQL or direct storage credentials.

Private Data Layer
SQL Database Cluster (users, classes, grades, attendance), NoSQL Database (messages, feeds, logs), Redis Cache (sessions, hot metadata, rate-limit counters), and Object Storage (files, videos, backups). A Database Security Group restricts inbound connections to App Layer IPs only.

Async Processing Layer
Message Queue (Kafka/RabbitMQ) buffers high-volume events. Worker Services consume these events for retries, bulk grading, file processing, and report generation. The Analytics Module aggregates data for dashboards and performance trends.

Security Design

Control	Description
WAF & DDoS Protection	Filters SQL injection, XSS, and volumetric attacks before they reach app servers.
API Gateway Rate Limiting	Prevents brute-force and abusive traffic from reaching the application layer.
JWT Authentication	Stateless token-based auth with short expiry and Redis-backed session invalidation.
RBAC	Teachers, students, and admins have discrete permission sets enforced at the service layer.
IAM Least Privilege	Each service and worker has only the minimum AWS/cloud permissions it needs.
DB Security Groups	Only App Layer IPs can reach databases; no public endpoint is exposed.
Encryption	TLS in transit; AES-256 at rest for databases and object storage.
Backup & DR	Scheduled snapshots of SQL, NoSQL, and object storage with cross-region replication.

4. Module Descriptions

Public Entry Layer

Internet Gateway	Single public entry point. Terminates external connections and routes to CDN.
CDN	Caches and serves static files (JS, CSS, images, videos). Reduces origin load by up to 90% for static assets.
WAF / DDoS Protection	Inspects all HTTP/S traffic. Blocks known attack signatures, rate-limits IPs, and filters bot traffic.
Application Load Balancer	Layer-7 HTTPS routing. Distributes sessions across App Server A and B with health-check failover.
API Gateway	Validates API tokens, enforces per-user rate limits, and routes to the correct internal service.

Security & Authentication

Auth Service	Handles login, token issuance (JWT), and token validation. Delegates SSO to external providers via OAuth 2.0.
RBAC Service	Checks the user's role (student/teacher/admin) against the requested action. Denies at the service boundary.
IAM	Cloud-level least-privilege policies for every service account and worker function.

EduManage Core Services

Classroom Service	Creates classrooms, manages enrollment, and records attendance. Writes to the SQL cluster and pushes events to the queue.
Assignment Service	Accepts file uploads, stores them in object storage, and writes submission metadata to SQL. Triggers grading workers via queue.
Grading Service	Validates scores, persists grades to SQL, and dispatches grade-notification events.
Communication Service	Handles chat messages, live-class events, and announcements. Persists to NoSQL and emits queue events for push delivery.
Notification Service	Consumes queue events and dispatches email, SMS, and push alerts through external provider APIs.
Resource Service	Serves study materials from object storage, checking CDN for cache hits first.

Data Storage Layer

SQL Database Cluster	Primary store for structured data: users, classrooms, enrollments, assignments, submissions, grades, and attendance. Supports read replicas.
NoSQL Database	Flexible-schema store for communication logs, activity feeds, and event records that change shape over time.
Redis Cache	In-memory store for JWT session data, hot entity metadata, and per-IP rate-limit counters. TTL-based eviction keeps data fresh.
Object Storage	Blob storage for assignment files, study materials, lecture videos, and backup snapshots.

Async Processing Layer

Message Queue (Kafka/RabbitMQ)	Durable event bus that decouples producers (services) from consumers (workers). Guarantees at-least-once delivery.
Worker Services	Stateless consumers that retry failed jobs, process bulk grading, transcode videos, and generate report exports.
Analytics Module	Aggregates grade distributions, attendance rates, and engagement metrics into dashboards for administrators.

DevOps & Observability

GitHub Actions CI/CD	Automated pipelines build, test, and push Docker images on every merge to main.
Container Registry	Versioned Docker images pulled by App Servers and Workers during deployment.
Monitoring & Logs	Centralised metrics, distributed traces, and structured logs. Alerts fire on latency spikes, error rates, or queue depth.
Backup & Disaster Recovery	Scheduled snapshots with cross-region replication. RTO < 4 hours; RPO < 1 hour.

5. Database Design

SQL Schema

PostgreSQL (or MySQL) stores all strongly-relational academic data. Tables use UUIDs as primary keys and foreign-key constraints to maintain referential integrity. Read replicas handle analytics queries.

Table	Key Columns
`users`	id (UUID PK), name, email, password_hash, role (ENUM: student/teacher/admin), institution_id, created_at, updated_at
`classrooms`	id (UUID PK), title, description, teacher_id (FK→users), institution_id, created_at
`enrollments`	id (UUID PK), classroom_id (FK), student_id (FK), enrolled_at, status
`assignments`	id (UUID PK), classroom_id (FK), title, description, due_date, file_key (object storage), created_at
`submissions`	id (UUID PK), assignment_id (FK), student_id (FK), file_key, submitted_at, status
`grades`	id (UUID PK), submission_id (FK), grader_id (FK), score, max_score, feedback, graded_at
`attendance`	id (UUID PK), classroom_id (FK), student_id (FK), session_date, status (present/absent/late)

NoSQL Collections

MongoDB (or DynamoDB) stores flexible-schema documents for communication and activity data whose shape may evolve independently of the SQL schema.

messages

```
{ _id, sender_id, classroom_id, content, type: 'chat'|'announcement', attachments[], timestamp }
```

activity_feed

```
{ _id, user_id, event_type, entity_id, entity_type, metadata{}, created_at }
```

notification_log

```
{ _id, recipient_id, channel: 'email'|'sms'|'push', content, status, sent_at }
```

Redis Cache

- **`session:{user_id}`** — JWT payload + expiry; TTL = token lifetime
- **`rate:{ip}`** — Request counter; TTL = 1 minute sliding window
- **`class_meta:{class_id}`** — Classroom metadata hot cache; TTL = 5 minutes
- **`online_users:{class_id}`** — Set of currently-online student IDs for live sessions

Object Storage

Amazon S3 (or equivalent) uses a structured key prefix scheme for efficient retrieval and lifecycle policies for cost management.

- **`submissions/{year}/{month}/{submission_id}/`** — Student assignment uploads
- **`resources/{classroom_id}/{file_name}`** — Study materials and slides
- **`videos/{classroom_id}/{session_id}/`** — Lecture recordings
- **`backups/{date}/`** — Nightly database snapshots

6. Technology Stack

Frontend

Component	Technology	Notes
Web App	**React 18 + Next.js 14 (App Router)**	SSR/SSG for fast initial load; TypeScript throughout.
Mobile App	**Flutter or React Native**	Single codebase targeting iOS and Android.
Admin Dashboard	**React + Recharts**	Real-time analytics charts and institution management.

Backend

Component	Technology	Notes
API Framework	**Python FastAPI**	Async-native, auto-generated OpenAPI docs, Pydantic validation.
Auth Library	**python-jose + Passlib**	JWT creation/validation; bcrypt password hashing.
ORM	**SQLAlchemy 2.0 + Alembic**	Async queries; schema migrations with version history.
Task Queue	**Celery + Redis broker**	Distributed task execution with retry and backoff.

Infrastructure

Component	Technology	Notes
Cloud	**AWS / GCP / Azure**	Multi-AZ deployment for high availability.
Containers	**Docker + Kubernetes (EKS/GKE)**	Rolling deployments; horizontal pod autoscaling.
CDN	**AWS CloudFront or Cloudflare**	Global edge caching; sub-10ms static asset delivery.
Queue	**Apache Kafka or RabbitMQ**	Durable, ordered event streaming for async workflows.

Data Storage

Component	Technology	Notes
SQL	**PostgreSQL 16**	ACID-compliant; read replicas for analytics; JSONB for hybrid data.

NoSQL	**MongoDB Atlas or DynamoDB**	Flexible schema; auto-scaling; global replication.
Cache	**Redis 7 (ElastiCache)**	In-memory key-value; Pub/Sub for real-time events.
File Storage	**Amazon S3**	11 nines durability; lifecycle policies; signed URLs.

Observability

Component	Technology	Notes
Metrics	**Prometheus + Grafana**	Custom dashboards for latency, error rates, and throughput.
Tracing	**OpenTelemetry + Jaeger**	Distributed trace context propagated across all services.
Logging	**Loki / CloudWatch Logs**	Structured JSON logs; correlated with trace IDs.
Alerting	**PagerDuty / Alertmanager**	On-call rotations triggered by SLO breaches.

CI/CD & Security

Component	Technology	Notes
CI/CD	**GitHub Actions**	Build → Test → Lint → Docker push → K8s rolling deploy.
Registry	**AWS ECR / GitHub Container Registry**	Signed and scanned images.
Secrets	**AWS Secrets Manager / HashiCorp Vault**	Zero-hardcoded credentials.
SAST	**Bandit (Python) + Trivy (containers)**	Shift-left security scanning in the pipeline.

7. Implementation Details

Component Flow Overview

The Python notebook *edumanage_component_flow.ipynb* models every architecture component as a pure function that accepts a request dictionary, makes routing decisions using if-else logic, appends to a trace log, and returns the mutated request to the next component.

Request Lifecycle

#	Step	Description
1	**Client sends request**	Internet users (student, teacher, admin) send HTTPS requests from browser or mobile app.
2	**Internet Gateway**	Single public entry point receives the connection.
3	**CDN Check**	If action is view_resource, CDN serves cached content directly. Otherwise forwards to WAF.
4	**WAF Filtering**	Inspects payload for blocked flags or injection patterns. Rejects and logs malicious requests.
5	**Load Balancer**	Routes by request ID parity: even → App Server A, odd → App Server B.
6	**API Gateway**	Validates action against allowed-action whitelist. Rejects unknown actions with 400.
7	**Auth Service**	Verifies JWT token. Login requests bypass token check. Invalid tokens return 401.
8	**RBAC Service**	Cross-checks user role against action permission matrix. Denied requests return 403.
9	**Service Router**	Maps action string to the correct microservice.
10	**Core Service**	Business logic executes: DB writes, file storage, grade validation, etc.
11	**Message Queue**	Heavy or notification events are pushed to Kafka/RabbitMQ for async processing.
12	**Worker / Notification**	Workers consume queue events; Notification service dispatches email/SMS/push.

RBAC Logic

Role-Based Access Control is enforced at the RBAC Service before any request reaches a business service. The permission matrix is:

Action	Student	Teacher	Admin

login	✓	✓	✓
create_class	✗	✓	✗
submit_assignment	✓	✗	✗
grade_assignment	✗	✓	✗
send_message	✓	✓	✗
view_report	✗	✓	✓
view_resource	✓	✓	✓

Async Worker Processing

After core services enqueue events, Worker Services consume them asynchronously. The notebook models this with a FIFO list representing the Kafka queue and a worker function that dequeues and processes each job.

Message Queue (enqueue)

```
def message_queue(request):
    job = {
        "job_id": make_id("job"),
        "action": request["action"],
        "payload": request["payload"]
    }
    MESSAGE_QUEUE.append(job)
    log(request, "Message Queue: job enqueued for async processing")
    request["next_component"] = "notification_service"
    return request
```

Worker Service (process)

```
def worker_service():
    if not MESSAGE_QUEUE:
        return {"status": "idle", "message": "No jobs in queue"}
    job = MESSAGE_QUEUE.pop(0)
    if job["action"] == "submit_assignment":
        OBJECT_STORAGE[f"processed_{job['job_id']}"] = "processed"
    return {"status": "completed", "job_id": job["job_id"]}
```

8. Screenshots & Diagrams

System Architecture Diagram

The Mermaid flowchart below (rendered from system-architecture-diagram.mmd) illustrates all components and their directional connections across the five architecture layers.

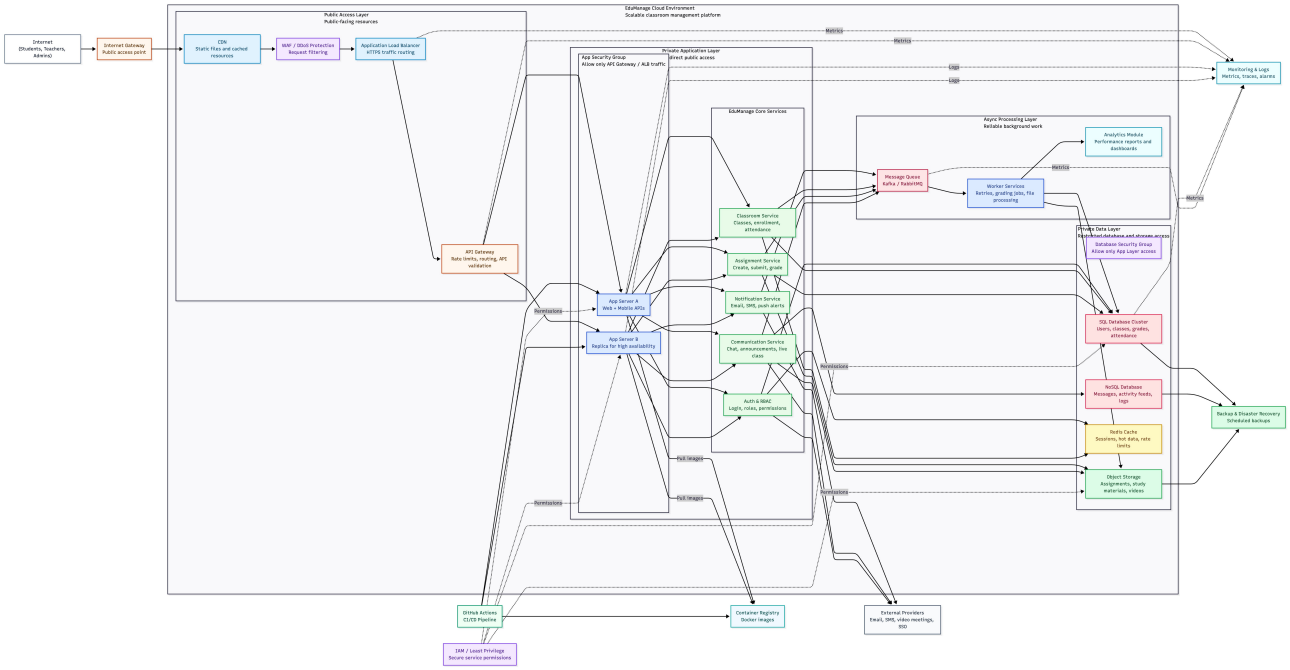


Figure 2 – Full System Architecture (Mermaid LR Flowchart)

Architecture Component Colour Legend

- Infrastructure (Internet Gateway, API Gateway)
- Load Balancer / CDN
- Compute (App Servers, Workers)
- Microservices (Classroom, Assignment, etc.)
- Databases (SQL, NoSQL)
- Cache (Redis)
- Message Queue (Kafka/RabbitMQ)
- Monitoring & Analytics
- Security (WAF, RBAC, IAM)
- External Providers (Email, SMS, SSO)

9. Future Scope

EduManage is designed for extensibility. The microservices architecture allows new capabilities to be added as independent services without disrupting existing functionality. The following areas represent high-value enhancements for future iterations.

AI-Powered Learning Assistance
Integrate a large language model service to provide personalised study recommendations, automatic feedback on essay submissions, intelligent tutoring chatbots, and plagiarism detection. Models can be fine-tuned on institution-specific curriculum data.

Live Virtual Classrooms
Add a WebRTC-based video conferencing module with screen sharing, interactive whiteboards, breakout rooms, and real-time attendance tracking. Integrate with Zoom or Google Meet APIs as a fallback for large cohorts.

Adaptive Assessments
Build a question-bank service that generates individualised test papers based on student performance history. Adaptive difficulty algorithms reduce cheating and improve learning outcomes.

Mobile Offline Support
Implement Progressive Web App (PWA) and mobile offline-first sync using IndexedDB and background sync APIs. Students in low-connectivity regions can download materials and submit assignments when connectivity is restored.

Multi-Tenancy & SaaS Model
Introduce tenant isolation at the database level (schema-per-tenant or row-level security) to enable a SaaS offering for multiple independent institutions on shared infrastructure with custom branding and data residency options.

Advanced Analytics & BI
Connect the analytics module to a full data warehouse (BigQuery, Redshift, or Snowflake) with dbt transformations, enabling institution-wide longitudinal studies, early-warning systems for at-risk students, and government reporting exports.

Gamification & Engagement
Add badges, leaderboards, streaks, and XP systems to increase student motivation. Integrate with the notification service to celebrate milestones and encourage consistent engagement.

Accessibility & Internationalisation
Full WCAG 2.1 AA compliance, screen-reader-optimised UI, multi-language support via i18n (Arabic RTL, Hindi, Spanish, etc.), and alternative content formats (audio transcripts, high-contrast themes).

Conclusion

EduManage demonstrates that a modern, cloud-native architecture can address the core challenges of digital education management at scale. The system's layered security, microservices decomposition, asynchronous processing, and polyglot persistence model provide a solid foundation for a production-grade platform. With the enhancements outlined above, EduManage can evolve into a comprehensive educational ecosystem serving millions of learners worldwide.

EduManage Classroom Management Platform – System Design Case Study
Repository: github.com/RohanVashisht1234/college-system-design
Colab: colab.research.google.com/drive/1BXZRrdZ3ZTJDe93QXP8ldVA0SFM0LkgC
This document is an academic demonstration. Not intended for production use.